

Undirected Connectivity in Log-Space (Omer Reingold)

Camille ORDRONNEAU, Joachim ROBERT

Index Terms—Notions étudiées : Théorie de la complexité, égalité de classes, Théorie des graphes, graphe non-orienté, régulier, graphe expenseur, algèbre linéaire ...



Abstract—Dans cette synthèse, nous présentons les résultats pratiques, numériques obtenus à partir d'algorithmes théoriques utilisés afin de résoudre le problème connu sous le nom de *st-connectivity* et plus largement encore le problème d'égalité de classes de complexité : $SL = L$. L'algorithme présenté dans le papier d'Omer REINGOLD [1] est logarithmique en espace et polynomial en temps.

Notre travail s'inscrit dans la reproductibilité des algorithmes théoriques présentés par Reingold. L'implémentation a été réalisée dans le langage Python.

1 INTRODUCTION

Rappels théoriques

Définition 1.1 : Classe L et SL

- Classe **L** : ensemble des algorithmes résolubles par une machine de Turing **déterministe** en espace logarithmique
- Classe **SL** : ensemble des algorithmes résolubles par une machine de Turing **symétrique** en espace logarithmique

L'un des nombreux problèmes en théorie de la complexité, était la question de l'égalité d'ensemble :

$$SL = L \quad (1)$$

L'objectif de notre travail

Avant la parution du papier d'Omer REINGOLD (2008), il était connu que $L \subset SL$. De plus, le problème dont la résolution est étudiée dans ce papier : *USTCON*, est un problème qui avait été prouvé comme SL -complet [2]. Seul subsistait la preuve de la proposition suivante :

$$USTCON \in L \quad (2)$$

afin de démontrer l'équation 1. L'algorithme permettant cette preuve est ce sur quoi porte l'article de recherche sur lequel nous avons travaillé.

Le problème USTCON

ENTREE : Un graphe G à N sommets

. s et t deux sommets de G

SORTIE : **VRAI** s'il existe un chemin de s à t dans G

. **FAUX** sinon

Une solution possible à ce problème est un parcours en profondeur du graphe donné en entrée, seulement cette solution, en complexité spatiale, est en $\Theta(N)$ et non en $\Theta(\log(N))$. Il faut donc étudier une autre approche, un autre algorithme, pour résoudre ce problème.

Avec un graphe ayant toutes ses composantes connexes de diamètre (plus grande distance entre deux sommets) logarithmique, il existe un algorithme permettant de vérifier si s et t sont connectés. Il suffit de parcourir tous les chemins de longueur valant le diamètre du graphe à partir de s . Si t n'est pas rencontré alors il ne peut pas appartenir à la même composante connexe que s et n'est donc plus généralement pas relié à s .

Tous les graphes ne possèdent pas forcément un diamètre logarithmique, par exemple les cycles de longueur N ont un diamètre de $\frac{N}{2}$. Il faut alors transformer notre graphe initial, en conservant chaque composante connexe, de sorte que chaque composante ait un diamètre logarithmique en la taille du graphe initial.

2 PRÉLIMINAIRES

Il est important avant de rentrer dans le coeur de notre expérience, de présenter quelques notions de théorie des graphes ainsi que des notations, qui serviront à la compréhension de ce qui va être exposé par la suite.

Définition 2.1 : Graphe D -régulier

Soit D un entier naturel. On appelle graphe D -régulier, tout graphe tel que chaque sommet de ce graphe possède D arêtes.

Définition 2.2 : Labellisation d'un graphe

Soit G un graphe. On parle de labellisation de G lorsque pour chaque sommet du graphe, on associe un indice unique pour toute arête sortante. Une arête peut avoir à chaque extrémité un label différent.

Définition 2.3 : Rotation Map [3]

Soit D, N deux entiers naturels, G un graphe non-orienté à N sommets, D -régulier. On appelle *Rotation Map* de G la fonction $Rot_G : [N] \times [D] \rightarrow [N] \times [D]$, telle que : $\forall (u, i), (v, j) \in [N] \times [D], Rot_G(u, i) = (v, j)$

Signifiant que l'arête labellisée par la lettre i pour le sommet u , est labellisée j pour le sommet v .

Remarque 2.4 : on écrira par la suite $G(N, D, \lambda)$, un graphe G , D -régulier à N sommets et pour lequel la seconde plus grande valeur propre (en valeur absolue) de la matrice d'adjacence de G divisée par D .

3 GRAPHE EXPENSEUR

L'objectif majeur de l'algorithme est d'effectuer des transformations sur le graphe initial afin d'augmenter son taux d'expansion. Le diamètre d'un graphe étant inversement proportionnel à son expansion, celui-ci diminuera, comme indiqué par la formule 3.

Definition 3.1 Graphe expenseur

Il existe plusieurs définitions d'un graphe expenseur [4]. Celle qui est choisie dans l'article dont nous étudions la mise en pratique nous dit qu'un graphe expenseur est un graphe avec un bon taux d'expansion.

Un bon taux d'expansion signifie intuitivement que pour chaque sous-ensemble de sommets de G de taille inférieure à $\frac{N}{2}$, il y a plus de noeuds dans le voisinage strict de cet ensemble, que de noeuds à l'intérieur de cet ensemble.

Il a été montré qu'une bonne manière d'estimer l'expansion d'un graphe est la seconde valeur propre de la matrice d'adjacence normalisée de ce graphe [5].

Théorème 3.2 :

Soit un graphe non-orienté $G(N, D, \lambda)$. Si $\lambda < 1$ Le diamètre Δ de G vérifie :

$$\Delta < 2 \lfloor \sqrt{\frac{2}{1-\lambda}} \log_2(N) \rfloor \quad (3)$$

Afin de majorer le diamètre d'un graphe régulier non-orienté par une borne logarithmique, on comprend donc par l'équation 3 que les transformations à effectuer sur le graphe doivent permettre de majorer λ par une constante. En l'état λ peut potentiellement être très proche de 1, ce qui rend cette borne supérieure inutilisable.

Dans le papier de recherche il a été décidé de chercher une transformation permettant de majorer λ par $\frac{1}{2}$. Cela nous donnerait un diamètre logarithmique : $\Delta < 4 \lfloor \log_2(N) \rfloor$

Mise à la puissance

Une bonne manière d'augmenter l'expansion d'un graphe est sa mise à la puissance. Imposant une augmentation du degré du graphe en gardant le nombre de sommets constant.

Théorème 3.3 :

Soit G un graphe (N, D, λ) et t un entier. Alors G^t sera un graphe (N, D^t, λ^t) . En effet, mettre à la puissance t un graphe consiste à mettre

à la puissance t la matrice d'adjacence de ce graphe.

Sur python, nous utilisons la bibliothèque `numpy` pour faire cette mise en puissance de matrice.

Mettre en puissance un graphe ne fait pas augmenter le nombre de sommets, mais ce n'est pas le cas de la prochaine transformation qui augmente la taille du graphe. Pour respecter la complexité spatiale logarithmique, on ne peut pas calculer entièrement les graphes sous la forme de matrice, il a donc fallu utiliser les *Rotation Map* ainsi que le théorème suivant :

Théorème 3.4 :

$Rot_{G^t}(v_0, (a_1, a_2, \dots, a_t)) = (v_t, (b_t, b_{t-1}, \dots, b_1))$ où pour tout i dans $\llbracket 0, t \rrbracket$ on a : $(v_i, b_i) = Rot_G(v_{i-1}, a_i)$

Dans la version finale de l'algorithme nous utilisons les *Rotation Maps*. Pour vérifier la correction de la mise en puissance, nous avons implémenté à la fois les transformations sous la forme de matrices et sous la forme de *Rotation Maps*.

Utiliser uniquement la puissance de graphe afin de faire diminuer λ n'est pas suffisamment efficace, il convient donc d'introduire une nouvelle transformation.

Produit zig-zag

La transformation de graphe que nous allons introduire doit avoir deux effets, le premier est de réduire le degré du graphe qui sera mis par la suite à une certaine puissance. Le second concerne l'expansion du graphe, la nouvelle transformation ne doit pas avoir un impact trop important sur cette dernière.

Dans l'article, deux transformations possibles sont présentées mais c'est le produit zig-zag qui est utilisé par la suite, nous allons donc seulement présenter celui-ci.

Définition 3.5 : Produit zig-zag

Soit G un graphe (N, D, λ) et H un graphe (D, d, α) où $d \ll D$. On définit le produit zigzag de G et H , noté $G \otimes H$, comme étant un produit de remplacement modifié.

Dans un premier temps, on remplace chaque sommet de G par une copie de H . Le nombre de sommet est alors multiplié par D . Puis on décrit une arête de ce nouveau graphe comme suit : il existe une arête entre deux sommets u et v de $G \otimes H$ si on peut aller de u à v en empruntant une arête de H (le *zig*), puis une arête de G , suivi d'une autre arête dans H (le *zag*).

Dans cette transformation nous avons d possibilités pour le *zig* et d possibilités pour le *zag*, le graphe résultant aura alors un degré de d^2

Afin de tester une première implémentation de notre produit zig-zag, nous nous sommes référés à l'exemple donné sur la page Wikipédia destinée à ce produit : Produit zig-zag.

Nous voyons que notre résultat présenté en figure 2 concorde avec celui de la page Wikipédia, que l'on peut voir en figure 1.

Pour avoir une deuxième confirmation de la justesse de notre implémentation du produit zig-zag, nous avons

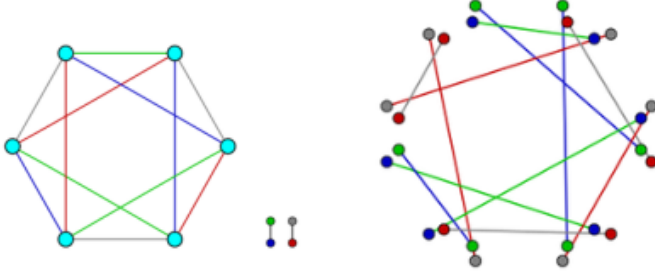


Fig. 1. Produit zig-zag sur wikipédia.

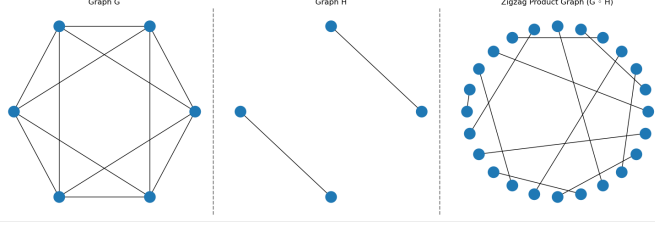


Fig. 2. Produit zig-zag avec notre algorithme.

utilisé la formule théorique suivante [3], donnant une borne supérieure précise sur la seconde valeur propre d'un graphe résultant d'un produit zig-zag entre deux graphes $G = (N, D, \lambda)$ et $H = (D, d, \alpha)$. Cela nous a permis ainsi de vérifier que nous obtenions des résultats corrects :

$$f(\lambda, \alpha) = \frac{1}{2}(1 - \alpha^2)\lambda + \frac{1}{2}\sqrt{(1 - \alpha^2)^2\lambda^2 + 4\alpha^2} \quad (4)$$

Après implémentation nous avons pu comparer la valeur théorique via la formule 4 et la valeur propre du graphe obtenue via le produit zig-zag. Nous avons, en prenant des graphes G 3-régulier aléatoires et le graphe cycle H 2-régulier à 3 sommets, pu lancer des tests sur des graphes de petite taille : $N < 20$.

De plus, toujours pour respecter la contrainte de complexité spatiale, nous avons ensuite implémenté de nouvelles fonctions utilisant les *rotation_map* via les formules suivantes :

- 1) $Rot_{G \times H}(v, a), (i, j) = ((w, b), (j', i'))$ où
- 2) $(a', i') = Rot_H(a, i)$
- 3) $(w, b') = Rot_G(v, a')$
- 4) $(b, j') = Rot_H(b', j)$

Avec la mise à la puissance et le produit zig-zag, nous avons deux transformations permettant de réduire la seconde valeur propre d'un graphe tout en gardant son degré constant. Nous pouvons donc passer à la transformation principale présentée dans l'article.

Transformation principale

Comme dit précédemment, $\tau(G, H)$, la transformation principale, reprend les deux transformations présentées précédemment. Elle est définie comme suit :

- 1) on pose $l = \sqrt{-\log_2(-\log_2(1 - \frac{1}{DN^2}))}$
- 2) on pose $G_0 = G$ et récursivement :

$$G_i = (G_{i-1} \otimes H)^8$$

- 3) On a $\tau_i(G, H) = G_i$ et $\tau(G, H) = G_l$

Afin de pouvoir effectuer un produit zigzag entre G_i et H , il faut que le degré de G_i soit égal au nombre de sommets de H . De plus il faut qu'à l'itération suivante, le produit zigzag soit encore possible donc le degré de G_i et G_{i+1} doivent être les mêmes. Comme le produit zigzag donne un graphe dont le degré vaut le degré de H au carré et que la puissance d'un graphe met le degré du graphe à la même puissance, on en déduit que H doit être un graphe D -régulier à D^{16} sommets et que G doit être D^{16} -régulier.

Il est prouvé dans l'article que si $\lambda(H) \leq 1/2$ alors $\lambda(\tau(G, H)) \leq 1/2$. Nous pouvons alors pour tout graphe G , dans un premier temps le transformer en graphe régulier, puis le transformer à l'aide d'un graphe H en un graphe expenseur. On notera par la suite G_{exp} tout graphe G auquel on aura effectué la transformation principale.

Afin de résumer l'algorithme ci-dessus, on peut dire qu'il prend en entrée deux graphes $G = (N, D^{16}, \lambda)$ et $H = (D^{16}, D, \frac{1}{2})$ et donne en sortie un graphe $G_{exp} = (N \cdot D^{16l}, D^{16}, \frac{1}{2})$. La taille du graphe final est beaucoup trop grande pour pouvoir le stocker en mémoire c'est la raison pour laquelle nous utilisons des *Rotations Maps* qui permettent de ne pas calculer entièrement ces graphes. Néanmoins cela ne nous permet pas de vérifier la correction de nos algorithmes car nous avons besoin de vérifier la valeur de $\lambda(G_{exp})$ ce qui nécessite la matrice d'adjacence de G_{exp} et donc son calcul et stockage complets.

Test de la transformation principale

La preuve de l'implication présentée précédemment se base principalement sur le fait que :

$$\forall i \in \llbracket 0, l \rrbracket \quad \lambda(G_{i+1}) < \lambda(G_i)^2$$

Pour tester l'algorithme, nous avons effectué une seule itération de la transformation principale, au lieu de l , pour comparer les valeurs propres obtenues. De cette manière nous avons un premier avis sur la correction de notre implémentation. Avoir un graphe initialement de degré D^{16} est nécessaire pour pouvoir enchaîner plusieurs itérations dans l'algorithme. Mais comme nous n'en faisons qu'une seule nous pouvons prendre seulement un graphe D -régulier.

Il nous fallait un graphe H tel que $\lambda(H) \leq \frac{1}{2}$, il se trouve que le graphe cycle sur 3 sommets est justement un tel graphe $(3, 2, \frac{1}{2})$. En partant de cela nous avons pu tester une itération de la boucle sur les graphes 3-régulier. Les graphes 3-régulier ont une bonne expansion, nous avons pu le remarquer car dès la première itération sur des graphes à 8 sommets, ils obtenaient une seconde valeur propre inférieure à $\frac{1}{2}$ alors qu'il aurait fallu $l = 8$ itérations normalement.

Nous avons pu pousser un peu plus l'étude :

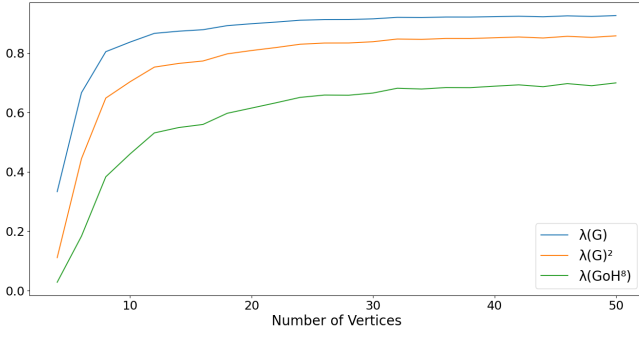


Fig. 3. Seconde valeur propre en fonction du nombre de sommet de G .

Nous avons testé une seule itération en prenant des graphes aléatoires 3-régulier à N sommets. En mesurant à chaque fois $\lambda(G)$, $\lambda(G)^2$ et $\lambda(GoH^8)$ et en moyennant sur 100 graphes de chaque taille on obtient le graphique 3.

On remarque, en moyenne, que $\lambda(GoH^8)$ est bien inférieure à $\lambda(G)^2$. Par exemple, il ne suffirait que de deux itérations sur un graphe de 50 sommets pour avoir $\lambda(G_2) \leq \frac{1}{2}$ au lieu de $l = 13$ itérations. Cela étant, il s'agit d'une moyenne et il peut y avoir des graphes ayant une expansion particulièrement mauvaise qui nécessiteraient plus d'itérations.

Ces tests ont été effectués pour vérifier la correction de la seconde valeur propre en sortie de l'algorithme. Maintenant, nous avons également cherché un moyen de vérifier la correction de $RotG_{exp}$.

G_{exp} n'est pas calculé entièrement mais Reingold ajoute en annexe du papier de recherche un pseudo-code permettant de calculer la *Rotation Map* de G_{exp} . Nous avons implémenté l'algorithme tel qu'il est écrit, et il ne fonctionne pas en tant que tel, il semble manquer le cas par défaut à prendre en compte. Il faut rajouter `else I <- I + 1` afin que l'algorithme soit réellement correct.

Mis à par cela, pour tester cette algorithme qui est bas niveau, nous avons ré-implémenté une version plus haut niveau basée sur la définition mathématique de la transformation. Nous avons obtenu des deux implémentations les mêmes résultats, nous en concluons que l'algorithme est correct.

Construction d'un graphe constant

Pour nos premiers tests, en prenant H valant le cycle à 3 sommets, nous avons pu réaliser une itération de la boucle de la transformation. Mais pour en réaliser plusieurs, il nous fallait un graphe $(D^{16}, D, \frac{1}{2})$. Un tel graphe existe et plusieurs constructions nous sont données dans des articles cités par Reingold. La constante D a un rôle dans la complexité temporelle de l'algorithme, c'est pourquoi nous avons recherché en dehors de notre article s'il y avait une construction utilisant $D = 3$.

Cependant, nous n'avons rien trouvé quant à cela. Nous avons donc utilisé une des constructions citée par Reingold et utilisant les corps premiers finis. La valeur minimale de

D ainsi obtenue est $D = 4489$. Néanmoins la *Rotation Map* de H en utilisant cette construction est calculable en temps raisonnable, de l'ordre de 64 opérations élémentaires.

Ainsi des exécutions de $RotG_{exp}$ en effectuant plusieurs itérations ont pu être effectuées. Mais nous n'avons pas pu exécuter entièrement l itérations. En effet, si on étudie plus précisément le nombre d'appels à $RotG$ est de 8 pour une itération à cause de la mise à la puissance 8. Au bout de l itérations il aura fallu 8^l appels à $RotG$ et un nombre équivalent d'appels à $RotH$.

4 USTCON

Le problème USTCON s'applique à tout graphe, mais les transformations évoquées jusqu'à maintenant ne concernent que les graphes régulier non-bipartite (i.e. non 2-coloriable).

Il existe un algorithme simple pour rendre un graphe régulier. Pour un graphe G à N sommets on remplace chaque sommet par un cycle de longueur N , on obtient $N \cdot N$ sommets. Puis, pour chaque sommet $(v, w) \in \llbracket 1, N \rrbracket^2$, il y a une arête entre (v, w) et (w, v) s'il y en a une entre v et w dans G . Sinon il y a une arête bouclant entre (v, w) et lui-même. On obtient ainsi un graphe 3-régulier à N^2 sommets. Par la suite on rajoute $D^{16} - 3$ arêtes bouclant sur chacun des sommets, le graphe obtenu, noté G_{reg} , est bien D^{16} -régulier.

Là aussi, la *Rotation Map* est utilisée afin de ne pas calculer entièrement le graphe. Par soucis de simplicité, et car cela n'impacte pas la conclusion finale, nous décidons d'abstraire cette transformation et de ne pas considérer N^2 au lieu de N dans le calcul de la complexité temporelle finale. En effet il existe d'autres transformations plus complexes, ne faisant pas augmenter autant le nombre de sommets.

Désormais, nous pouvons transformer tout graphe G D^{16} -régulier en un graphe de même degré avec une bonne expansion c'est-à-dire avec une seconde valeur propre inférieure à $1/2$ et donc un diamètre logarithmique.

En effet, on avait via la formule 3 une inégalité impliquant le diamètre Δ et λ . Or, après transformation on a $\lambda \leq 1/2$ donc $\sqrt{\frac{2}{1-\lambda}} \leq 2$ d'où un diamètre $\Delta \leq 4\log_2(|G|)$ avec $|G|$ le nombre de sommet du graphe G . Ainsi dans notre cas on a : $\Delta \leq 4\log_2(ND^{16l})$.

L'algorithme A_{exp} présenté dans l'article pour résoudre le problème USTCON est le suivant :

- 1) ENTREE : G_{exp}, s, t
- 2) On part de s
- 3) On énumère tous les chemins de longueur Δ
- 4) SORTIE : **VRAI** si t est atteint, **FAUX** sinon.

Un chemin est une suite de labels appartenant à $\llbracket 1, D^{16} \rrbracket$. Si nous voulons parcourir tous les chemins de longueur Δ , il faudra en parcourir $(D^{16})^\Delta$. De plus pour chaque transition entre deux sommets il faudra effectuer un appel à $RotG_{exp}$ de complexité 8^l . La complexité totale est alors :

$$8^l (D^{16})^\Delta$$

On peut remplacer Δ par $4\log_2(ND^{16l})$ et l par $\log_2(DN^2)$ en effectuant un développement à l'ordre 1 de $\log(1-x)$ avec x petit.

Ainsi, si l'on applique l'algorithme présenté ci-dessus la complexité temporelle est de l'ordre de :

$$8\log_2(DN^2)(D^{16})^{4\log_2(ND^{16\log_2(DN^2)})} \quad (5)$$

itérations.

Après simplification on obtient :

$$D^3 N^6 (ND^{16\log_2(DN^2)})^{64\log_2(D)}$$

Nous avons utilisé $D = 4489$, néanmoins pour se donner une borne minimale dans l'hypothèse où il existerait une construction de H plus efficace, nous allons prendre $D = 4$. On obtient alors :

$$2^{16390} N^{16518}$$

On comprend de par nos calculs, qu'il est impossible de tester complètement l'algorithme USTCON présenté dans l'article. C'est pourquoi jusqu'ici nous avons testé indépendamment chaque brique de la transformation sur de petits graphes. La reproduction identique, en respectant une complexité spatiale logarithmique, est compromise.

Néanmoins la complexité spaciale est bien respectée. Dans l'implémentation que nous avons faite, les seuls endroits où N apparaît est dans la construction d'une liste de taille l d'éléments de $\llbracket 1, 16 \rrbracket$ et dans le stockage d'un compteur pour énumérer $D^{16\Delta}$ chemins. Ainsi en prenant $D = 4$ la complexité spatiale déduite est :

$$8326\log_2(N) + 8198$$

Cela correspond bien à une complexité logarithmique et c'est un espace qui aujourd'hui est allouable physiquement sur une machine.

5 UTS & UXS

Dans son article, Reingold profite de son étude sur le problème USTCON pour utiliser ses résultats afin de construire ce qu'on appelle des *universal-traversal sequences*.

Une *universal-traversal sequences* ou UTS est une suite de labels, telle que pour n'importe quel graphe G non-orienté régulier labellisé et pour n'importe quel noeud de départ, en suivant cette suite on visite tous les noeuds du graphe d'entrée. On sait qu'une telle suite de label existe [6].

Reingold s'est rendu compte que sa transformation d'un graphe en un graphe expenseur est applicable pour tout graphe. De plus lors du parcours de tous les chemins de longueur Δ , peu importe le sommet de départ, on visitera tous les sommets de la même composante connexe car ils se trouvent tous à une distance inférieure à Δ du sommet de départ. Mais pour que cela fonctionne il faut revenir au sommet initial à chaque itération en effectuant le chemin inverse. Cela n'est pas possible dans le cas général, il faut une labélisation spécifique. La transformation en graphe régulier énoncé au début de la partie 4 permet une telle labélisation.

Ainsi en ne tenant uniquement compte que des labels, il est possible de construire une UTS.

Concernant les Universal Exploration Sequence, notées UXS, il s'agit de séquence d'offsets. Tout comme pour les UTS, en suivant une UXS on visite tous les noeuds d'un graphe de taille N , et cela peu importe le graphe. Mais on ne suit plus une suite de labels définie. On commence par un label quelconque, puis on incrémente le numéro du label par l'offset donné par l'UXS, modulo le degré du sommet sur lequel on se trouve. Cela permet d'en faire un outil applicable à des graphes non-régulier. La complexité temporelle de telles constructions à fait que nous n'avons pas pu nous pencher plus en détail sur celles-ci.

6 CONCLUSION

Reingold démontre dans son article que la *st-connectivity* peut se résoudre en espace logarithmique via la construction et la recherche de chemins dans un graphe expenseur. Seulement, il a été montré que bien qu'elle soit polynomiale, la complexité temporelle de la recherche d'un chemin liant deux sommets d'un graphe expenseur construit selon la mise en puissance et le produit zig-zag est bien trop importante pour pouvoir être testée convenablement.

Néanmoins, il a été possible de tester les composantes de la transformation principale ainsi que certaines étapes présentées dans le papier en gardant une complexité spatiale logarithmique, de manière à tester la reproductibilité de l'article de recherche.

Etant donné que cette manière de faire est suivie afin de démontrer l'égalité d'ensemble $SL = L$, aucun commentaire n'est fait quant à ce souci de complexité temporelle. Dans une optique de reproductibilité, il pourrait être intéressant de trouver un compromis, entre la complexité spatiale et temporelle. De sorte que cette dernière soit plus abordable et que son homologue spatiale ne s'éloigne pas trop d'une complexité logarithmique.

L'intégralité du code produit est disponible sur GitHub : **Reingold_Expander**

REFERENCES

- [1] Undirected Connectivity in Log-Space 2008
- [2] Symmetric space-bounded computation. Theor. Comput. Sci., 19:161–187, 1982
- [3] Omer Reingold, Salil Vadhan, and Avi Wigderson. Entropy waves, the zig-zag graph product, and new constant-degree expanders. Annals of Mathematics, 155(1), January 2002. Extended abstract in Proc. of FOCS '00
- [4] Omer Reingold, Luca Trevisan, and Salil P. Vadhan. Pseudorandom walks on regular digraphs and the rl vs. l problem. In Jon M. Kleinberg, editor, STOC, pages 457–466. ACM, 2006.
- [5] u"Isoperimetric Inequalities for Graphs, and Superconcentrators"
- [6] Romas Aleliunas, Richard M. Karp, Richard J. Lipton, László Lovász, and Charles Rackoff. Random walks, universal traversal sequences, and the complexity of maze problems. In 20th Annual Symposium on Foundations of Computer Science, pages 218–223, San Juan, Puerto Rico, 29–31 October 1979. IEEE